Lab3

Lab exercise 3 --- Signals and Systems

Reference book:

John R.Buck, Michael M. Daniel, Andrew C. Singer. Computer Exploration in Signals and Systems —— Using MATLAB.

Objective:

1. Perform the Fourier serises using MATLAB
2. Perform the Fourier transform using MATLAB. Plot the spectrum of signals.
3. Further understand the properties of Fourier transform.
4. Apply Fourier transform in modulation.

Related commands:

fft, fftshift, freqz, freqs, filter, residue, lsim

MATLAB exercises:

1. Compute the Discrete-time frequency response with freqz. 3.2
2. Eigenfunctions of Discrete-Time LTI Systems. 3.4(a)(b)(c).
3. Properties of the Continuous-Time Fourier Transform. 4.3(a)-(f)
4. Perform Partial Fraction Expansion. 4.5(b)
5. Amplitude Modulation with Morse code. 4.6(a)(b)(c)(d)

Report requirements:

For all of the exercises, please include your MATLAB code. We expect a formal lab report, it is important that you present your results clearly and in an organized manner.

1. Compute the Discrete-time frequency response with freqz. 3.2

3.2

Code:

```
a1=[1 -0.8];

b1=[2 0 -1];

[H1 omega1]=freqz(b1,a1,4)

[H2 omega2]=freqz(b1,a1,4,'whole')
```

Output:

```
> lab3_3_2


H1 =

   5.0000 + 0.0000i
   2.8200 - 1.3705i
   1.8293 - 1.4634i
   0.9258 - 0.9732i


omega1 =

         0
    0.7854
    1.5708
    2.3562


H2 =

   5.0000 + 0.0000i
   1.8293 - 1.4634i
   0.5556 + 0.0000i
   1.8293 + 1.4634i


omega2 =

         0
    1.5708
```

```
    3.1416
    4.7124

>>
```

Explanation:

[H1 omega1]=freqz(b1,a1,4) calculates the frequency response of the system at 4 equally spaced points between 0 and π

[H2 omega2]=freqz(b1,a1,4,'whole') is similar to the previous call, but it evaluates the frequency response at 4 equally spaced points between 0 and 2π.

2. Eigenfunctions of Discrete-Time LTI Systems. 3.4(a)(b)(c).

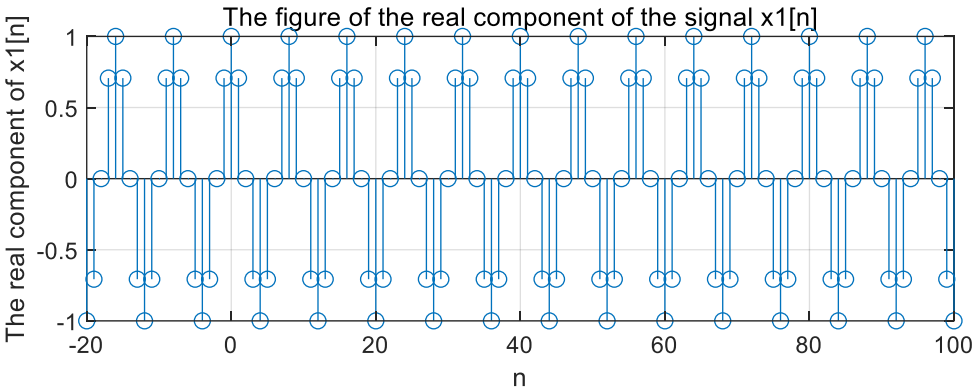
3.4.a

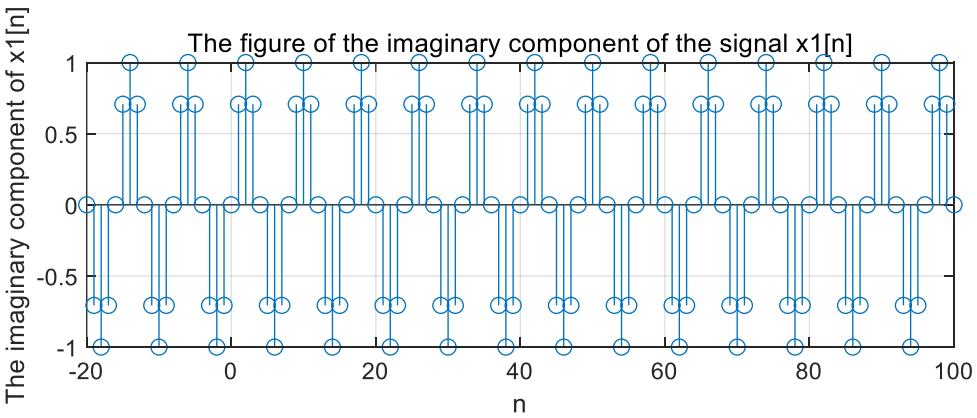
Code:

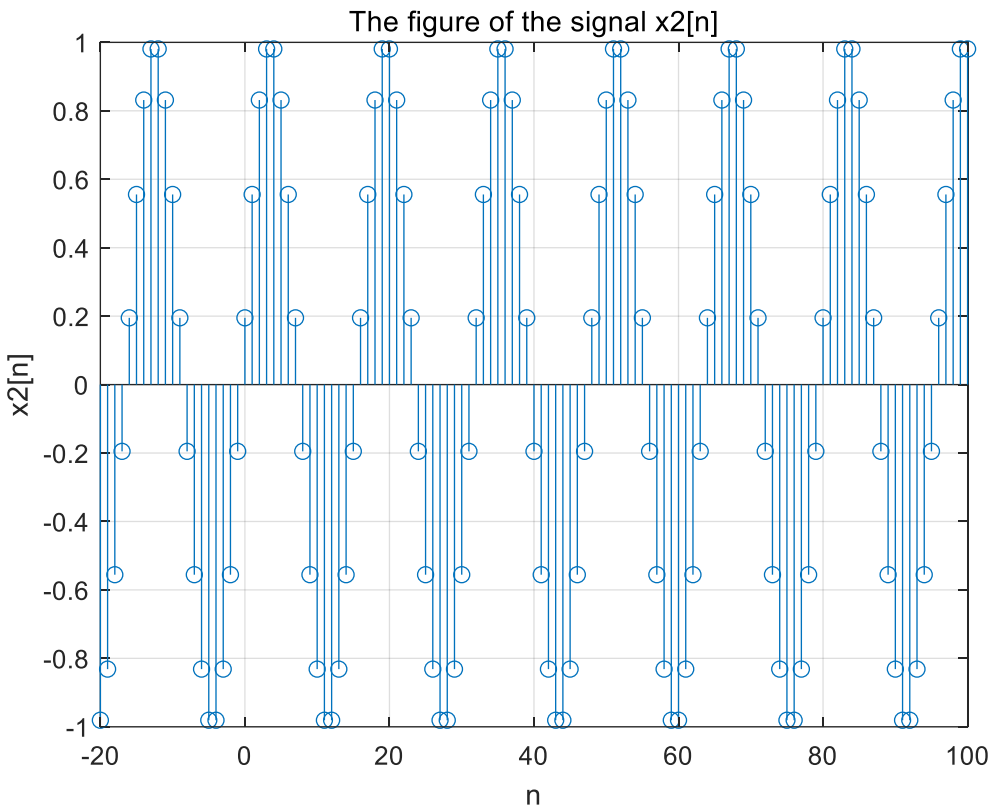
```
clc
clear all
n=-20:100;
x1=exp(1i*pi*n/4);
x2=sin(pi*n/8+pi/16);
x3=(9/10).^n;
x4=n+1;
figure(1);
subplot(2,1,1);
stem(n,real(x1));
xlabel('n');
ylabel('The real component of x1[n]');
title('The figure of the real component of the signal x1[n]');
grid on;
subplot(2,1,2);
stem(n,imag(x1));
xlabel('n');
ylabel('The imaginary component of x1[n]');
```

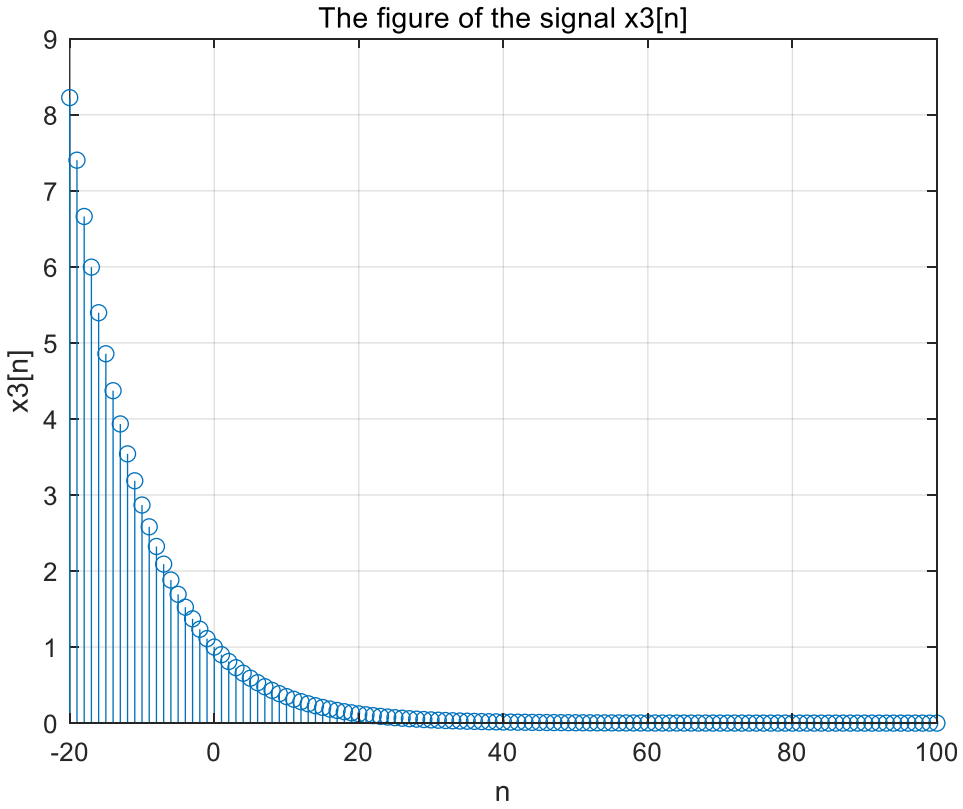
```
title('The figure of the imaginary component of the signal x1[n]');
grid on;
figure(2);
stem(n,x2);
xlabel('n');
ylabel('x2[n]');
title('The figure of the signal x2[n]');
grid on;
figure(3);
stem(n,x3);
xlabel('n');
ylabel('x3[n]');
title('The figure of the signal x3[n]');
grid on;
figure(4);
stem(n,x4);
xlabel('n');
ylabel('x4[n]');
title('The figure of the signal x4[n]');
grid on;
```

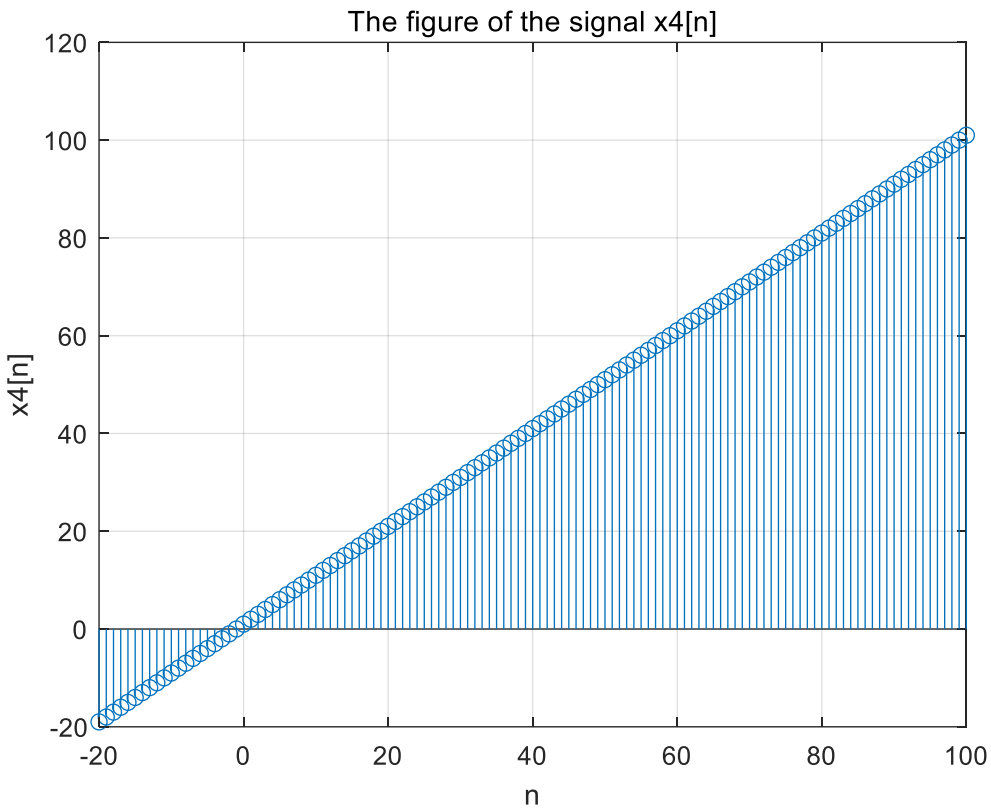
output:






The figure of the signal x2[n]
x2[n]
n
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
-20
0
20
40
60
80
100


The figure of the signal x3[n]
x3[n]
n
9
8
7
6
5
4
3
2
1
0
-20
0
20
40
60
80
100



3.4.bc

Code:

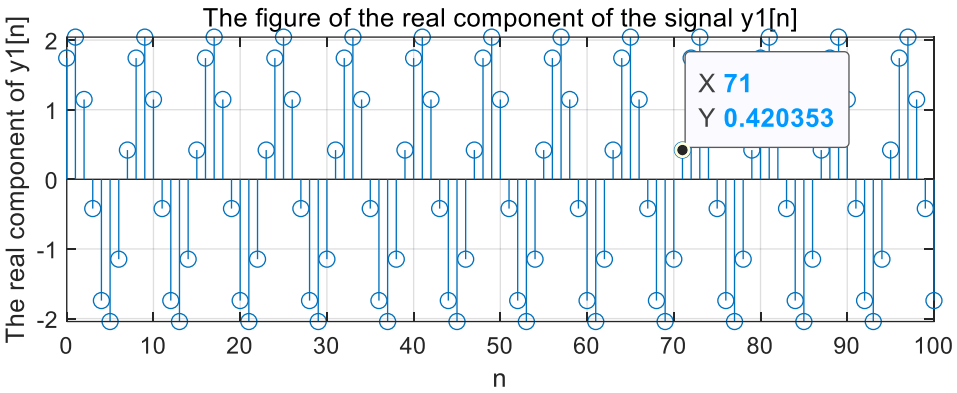
```
clc
clear all
n=-20:100;
a=[1 -0.25];
b=[1 0.9];
x1=exp(1i*pi*n/4);
x2=sin(pi*n/8+pi/16);
x3=(9/10).^n;
x4=n+1;
y1=filter(b,a,x1);
y2=filter(b,a,x2);
y3=filter(b,a,x3);
y4=filter(b,a,x4);
figure(1);
Y1=[];
for k=0:100
Y1(end+1)=y1(k+21);
end
subplot(2,1,1);
stem([0:100],real(Y1));
```

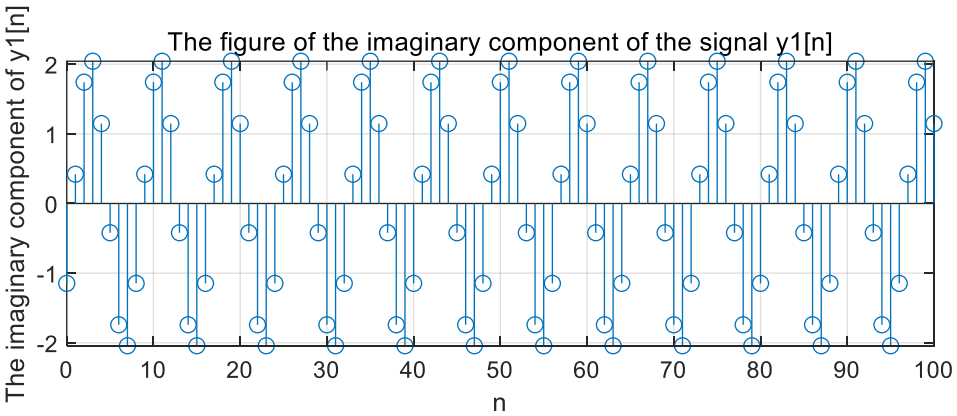
```
xlabel('n');
ylabel('The real component of y1[n]');
title('The figure of the real component of the signal y1[n]');
grid on;
subplot(2,1,2);
stem([0:100],imag(Y1));
xlabel('n');
ylabel('The imaginary component of y1[n]');
title('The figure of the imaginary component of the signal y1[n]');
grid on;
figure(2);
Y2=[];
for k=0:100
Y2(end+1)=y2(k+21);
end
stem([0:100],Y2);
xlabel('n');
ylabel('y2[n]');
title('The figure of the signal y2[n]');
grid on;
figure(3);
Y3=[];
for k=0:100
Y3(end+1)=y3(k+21);
end
stem([0:100],Y3);
xlabel('n');
ylabel('y3[n]');
title('The figure of the signal y3[n]');
grid on;
figure(4);
Y4=[];
for k=0:100
Y4(end+1)=y4(k+21);
end
stem([0:100],Y4);
xlabel('n');
ylabel('y4[n]');
title('The figure of the signal y4[n]');
grid on;
%.........(c)..............%
X1=[];
for k=0:100
X1(end+1)=x1(k+21);
```

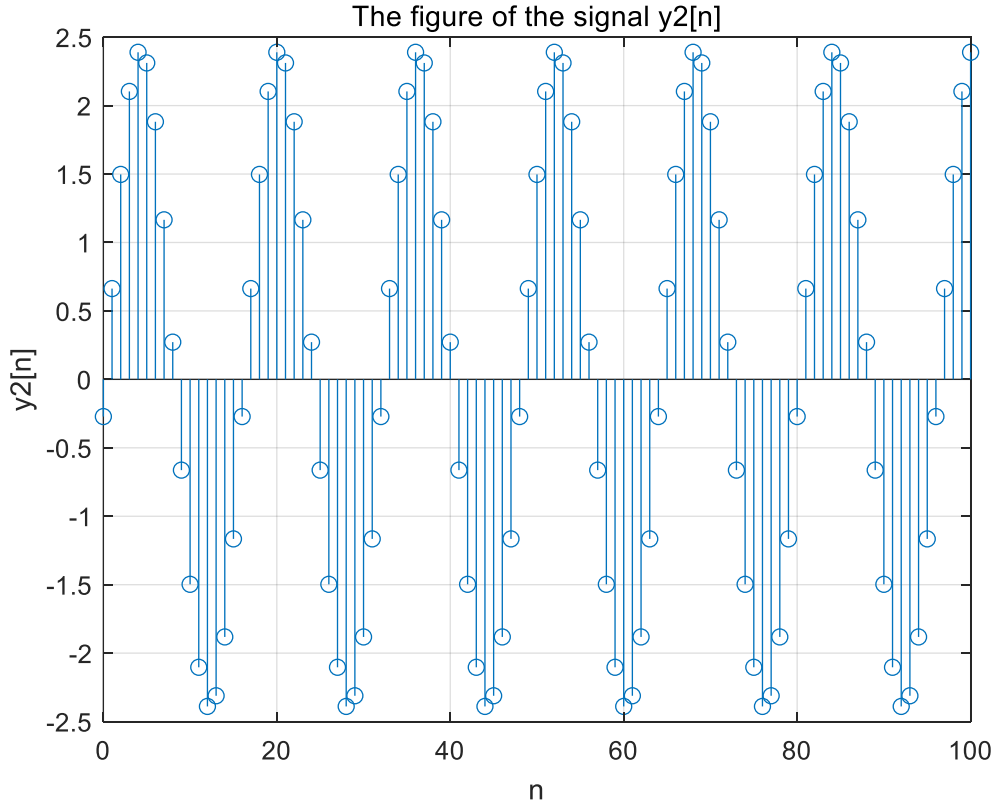
```
end
X2=[];
for k=0:100
X2(end+1)=x2(k+21);
end
X3=[];
for k=0:100
X3(end+1)=x3(k+21);
end
X4=[];
for k=0:100
X4(end+1)=x4(k+21);
end
H1=Y1./X1;
H2=Y2./X2;
H3=Y3./X3;
H4=Y4./X4;
figure(5);
subplot(2,1,1);
stem([0:100],real(H1));
xlabel('n'),ylabel('The real component of H1');
title('The figure of the real component of the vector H1');
grid on;
subplot(2,1,2);
stem([0:100],imag(H1));
xlabel('n'),ylabel('The imaginary component of H1');
title('The figure of the imaginary component of the vector H1');
grid on;
figure(6);
stem([0:100],H2);
xlabel('n');
ylabel('H2');
title('The figure of the vector H2');
grid on;
figure(7);
stem([0:100],H3);
xlabel('n');
ylabel('H3');
title('The figure of the vector H3');
grid on;
figure(8);
stem([0:100],H4);
xlabel('n');
ylabel('H4');
```

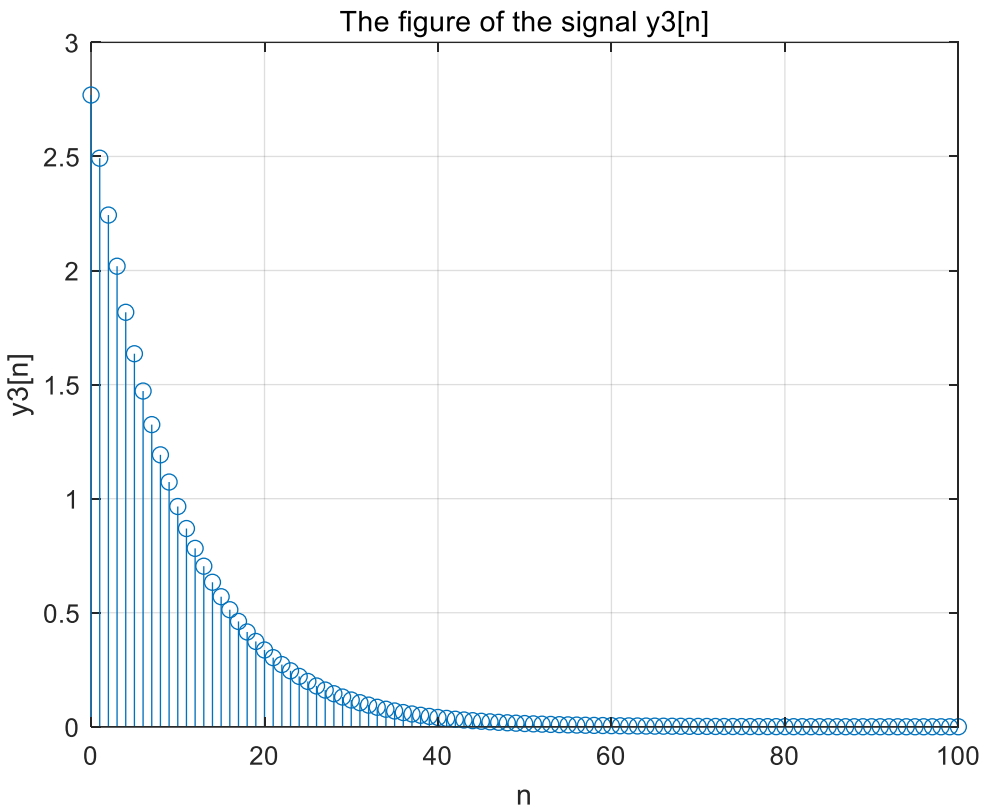
```
title('The figure of the vector H4');
grid on;
```

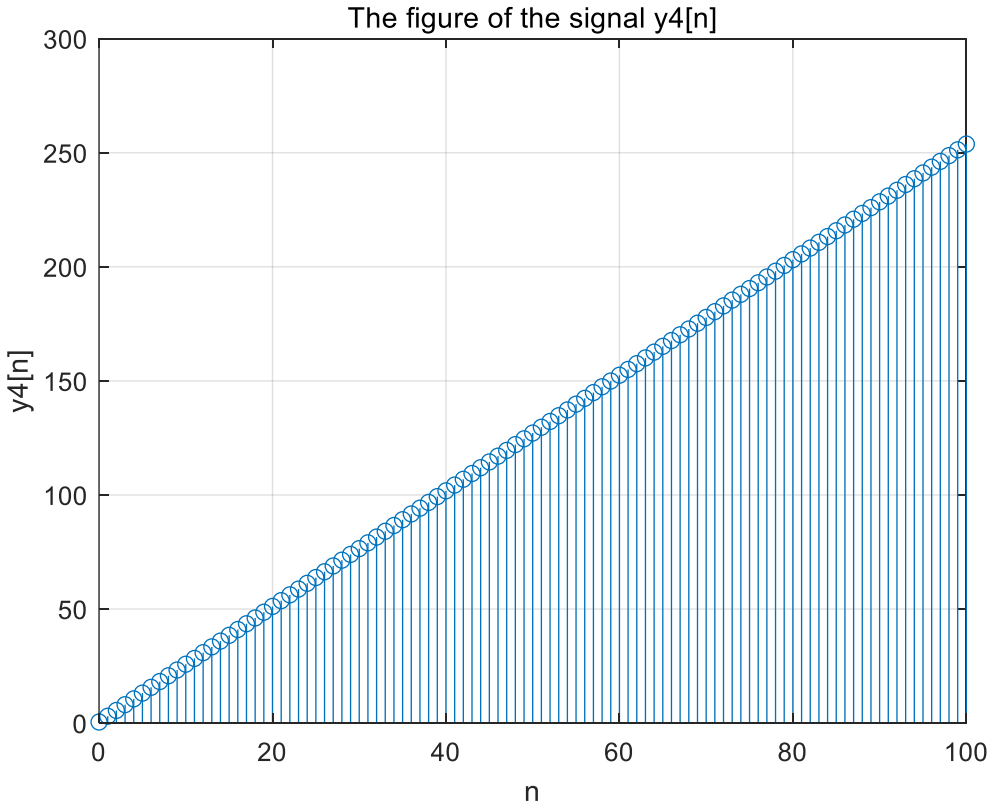
Output:

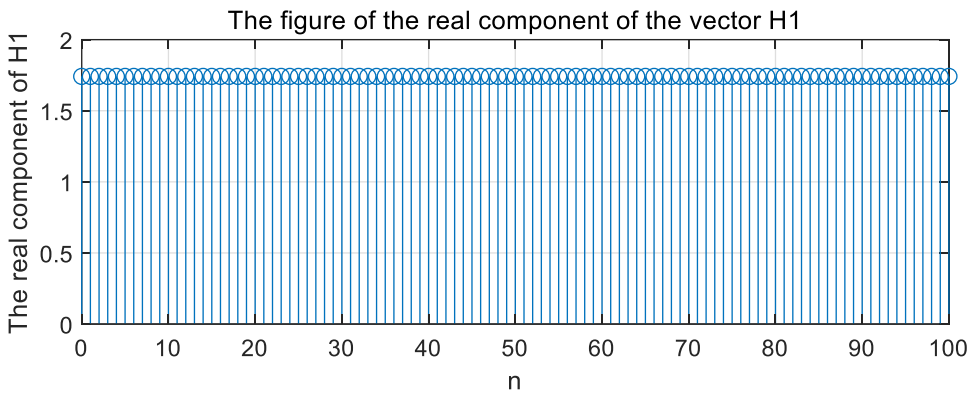


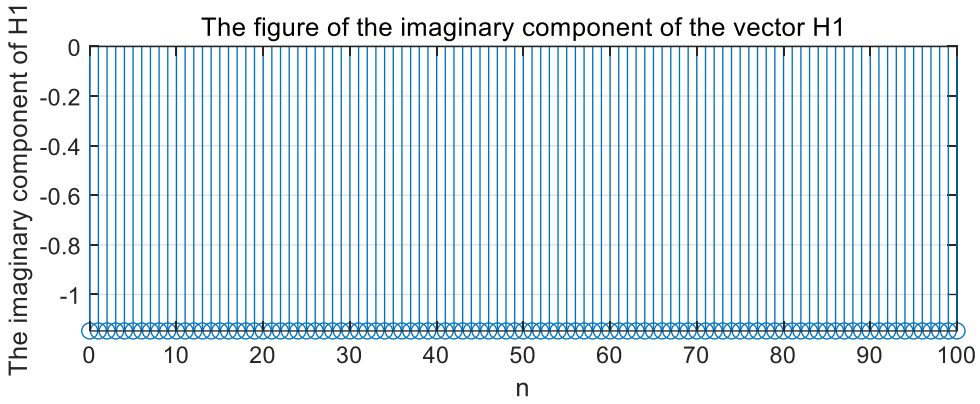


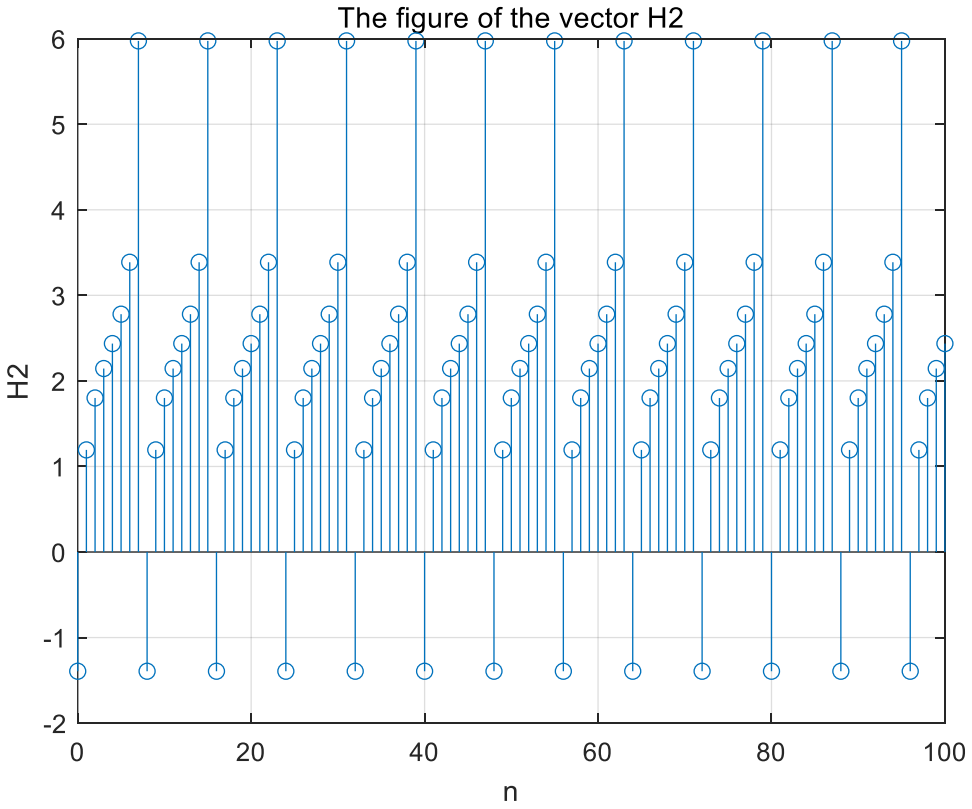


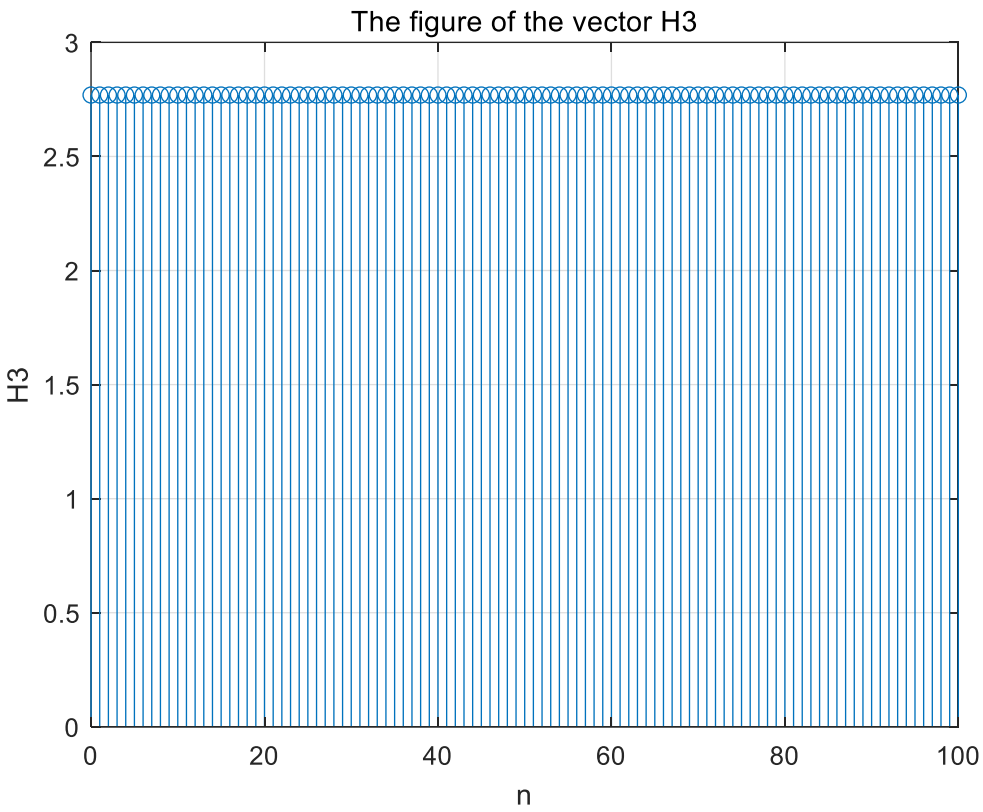

The figure of the signal y3[n]
y3[n]
n
0
0.5
1
1.5
2
2.5
3
0
20
40
60
80
100

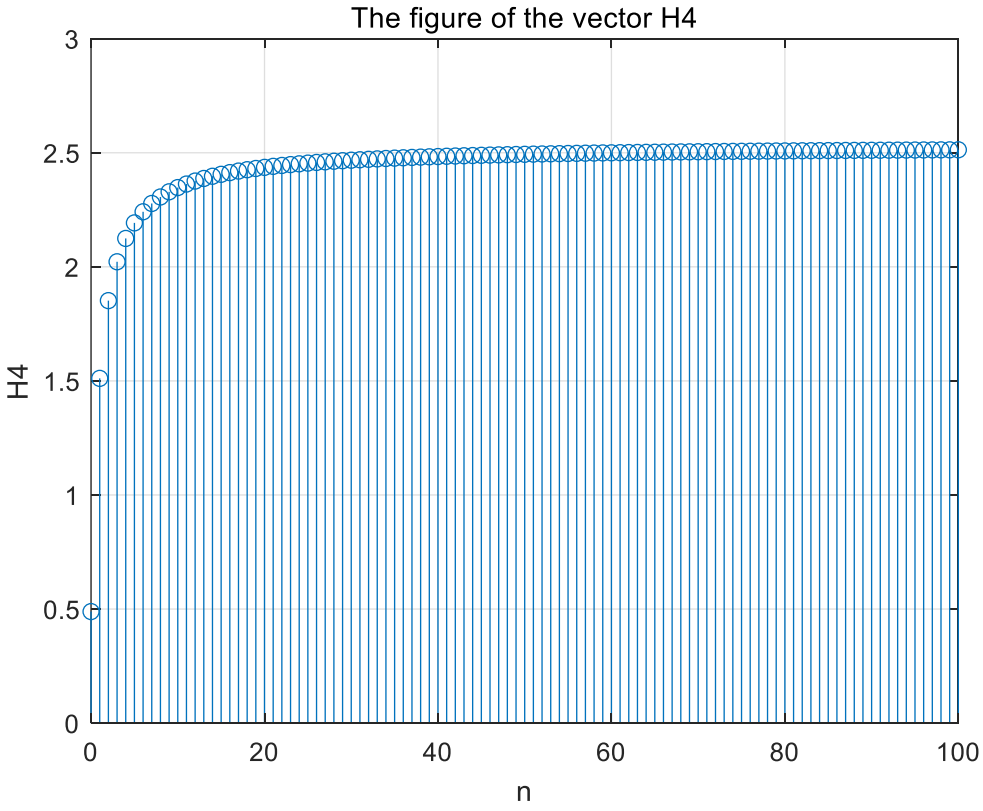

The figure of the signal y4[n]
y4[n]
n
0
50
100
150
200
250
300
0
20
40
60
80
100


The figure of the real component of the vector H1
The real component of H1
n


The figure of the imaginary component of the vector H1
The imaginary component of H1
n


The figure of the vector H2
H2
n


The figure of the vector H3
H3
n
0
0.5
1
1.5
2
2.5
3
0
20
40
60
80
100


The figure of the vector H4
H4
n
0
0.5
1
1.5
2
2.5
3
0
20
40
60
80
100

Explanation:

b)From the above figures, I found that:

1. Signal y1[n] (Real and Imaginary Parts):
    - From the plots, the real and imaginary parts of y1[n] are periodic signals, maintaining the same frequency characteristics as the input x1[n]=$e^{j(\pi/4)n}$
    - The output matches the form of the input, indicating that x1[n] is an eigenfunction of the system.
2. Signal y2[n]:
    - The output signal y2[n] is also a periodic signal, consistent with the basic frequency characteristics of the input x2[n]=sin(πn/8+π/16).
    - Since a sine wave can be decomposed into the sum of two complex exponential signals, and the system is LTI, the output preserves the input's frequency characteristics, indicating x2[n] is also an eigenfunction of the system.
3. Signal y3[n]:

- The output signal y3[n] is an exponentially decaying signal, retaining the same form as the input x3[n]= $(9/10)^n$
- The output matches the input form, confirming that x3[n] is an eigenfunction of the system.

4. **Signal y4[n]:**
   - The output signal y4[n] is a linearly increasing signal, which does not match the eigenfunction definition (the output of an eigenfunction should retain the same form as the input, with only amplitude and phase changes).
   - Therefore, x4[n]=n+1 is not an eigenfunction of the system.

Conclusion:

- **Eigenfunctions**: x1[n] x2[n] x3[n]
- **Not an Eigenfunction**: x4[n]

C)

1. **Real and Imaginary Components of H1 (Input: x1[n]= $e^{j(\pi/4)n}$:**

- **Real Component**: The plot of the real part of H1 is constant and stabilizes around a value (approximately 2).
- **Imaginary Component**: The imaginary part of H1 is constant and stabilizes around a value of approximately -1.
- **Conclusion**: Since H1[n] is constant (both real and imaginary parts), x1[n] is an **eigenfunction** of the system, and its eigenvalue is H1=2−j.

2. **H2[n] (Input: x2[n]=sin (πn/8+π/16)**: The plot of H2[n] shows oscillations and is not constant.

- **Conclusion**: x2[n] is **not an eigenfunction** of the system since H2[n] does not stabilize to a constant value.

3. **H3[n] (Input: x3[n]=(9/10)ⁿ**:

- The plot of H3[n] stabilizes to a constant value (2.5).
- **Conclusion**: x3[n]x_3[n]x3[n] is an **eigenfunction** of the system, and its eigenvalue is approximately H3=2.5

4. **H4[n]**:

- The plot of H4[n] shows a non-constant, increasing trend, which stabilizes at later points but does not remain constant for all values of n.
- **Conclusion**: x4[n] is **not an eigenfunction** of the system because H4[n] does not remain constant over the entire range.

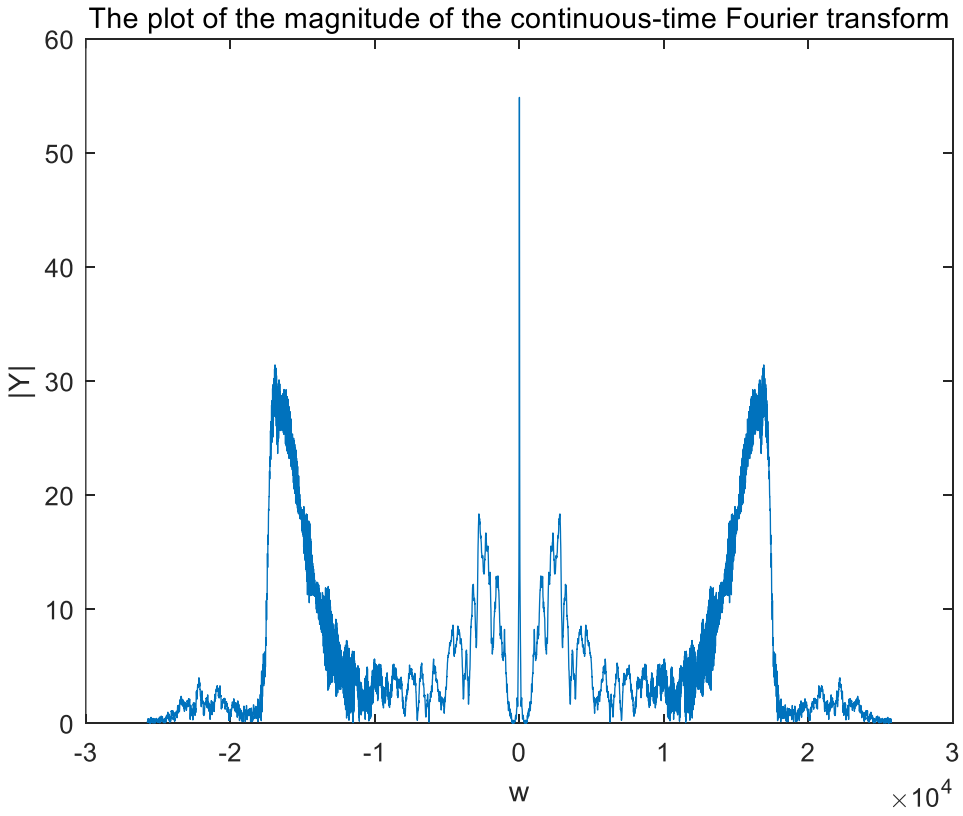
3. Properties of the Continuous-Time Fourier Transform. 4.3(a)-(f)

4.3a)b)

Code:

```
clc
clear all
load splat
y=y(1:8192);
N=8192;
fs=8192;
Y=fftshift(fft(y));
w=[-pi:2*pi/N:pi-pi/N]*fs;
plot(w,abs(Y));
xlabel('w');
ylabel('|Y|');
title('The plot of the magnitude of the continuous-time Fourier transform');
Y1=conj(Y);
y1=ifft(fftshift(Y1));
y1=real(y1);
sound(y, fs);
sound(y1,fs)
```

Output:



Explanation:

b) Because y(t) is real, $Y[-j\varpi]=Y^*[j\omega]$,

So if y(t) Fourier transform into $Y[j\omega]$,
Then $Y^*[j\omega] = Y[-j\omega]$ its reverse Fourier transform is equal to y(-t).
Therefore, we can hear the reverse vertion of y(t).

4.3c)

Explanation:

For y2(t), $Y2(j\omega) = |Y(jw)|$, so

$Y2(-j\omega) = |Y(-jw)| = |Y(jw)|$

$Y2^*(j\omega) = |Y*(jw)| = |Y(jw)|$

$Y2(-j\omega) = Y2^*(j\omega)$

Therefore, y2(t) is real

For y3(t),

$Y3(-j\omega) = e^{j\phi(-\omega)}$

$Y3^*(j\omega) = e^{j\phi(-\omega)}$

$Y3(-j\omega) = Y3^*(j\omega)$

Therefore, y3(t) is real

4.3d)

Code:

```
clc
clear all
load splat
y=y(1:8192);
N=8192;
fs=8192;
Y=fftshift(fft(y));
w=[-pi:2*pi/N:pi-pi/N]*fs;
Y2=abs(Y);
y2=ifft(fftshift(Y2));
y2=real(y2);
sound(y2,fs)
```

4.3e)

Code:

```
clc
clear all
load splat
y=y(1:8192);
N=8192;
fs=8192;
Y=fftshift(fft(y));
w=[-pi:2*pi/N:pi-pi/N]*fs;
Y3=exp(j.*phase(Y));
y3=ifft(fftshift(Y3));
y3=real(y3);
sound(y3,fs)
```

explanation

the sound in d) did not sound like y(t), however, the sound in f) was similar to y(t),despite the amplitude of sound .

Therefore, I think phase is more important in Fourier transform.

4. Perform Partial Fraction Expansion. 4.5(b)

Code:

```
clc
clear all
```

```
b1=[1 -2];
a1=[1 3/2 1/2];
[r1,p1]=residue(b1,a1)
```

Output:

```
r1 =


6

-5

p1 =


-1.0000

-0.5000
```

Explanation:

According to output,

$H1(j\omega) = \frac{6}{j\omega+1} + \frac{-10}{2j\omega+1} = \frac{2j\omega-4}{(j\omega+1)(2j\omega+1)}$

5. Amplitude Modulation with Morse code.

4.6(a)(b)(c)(d)

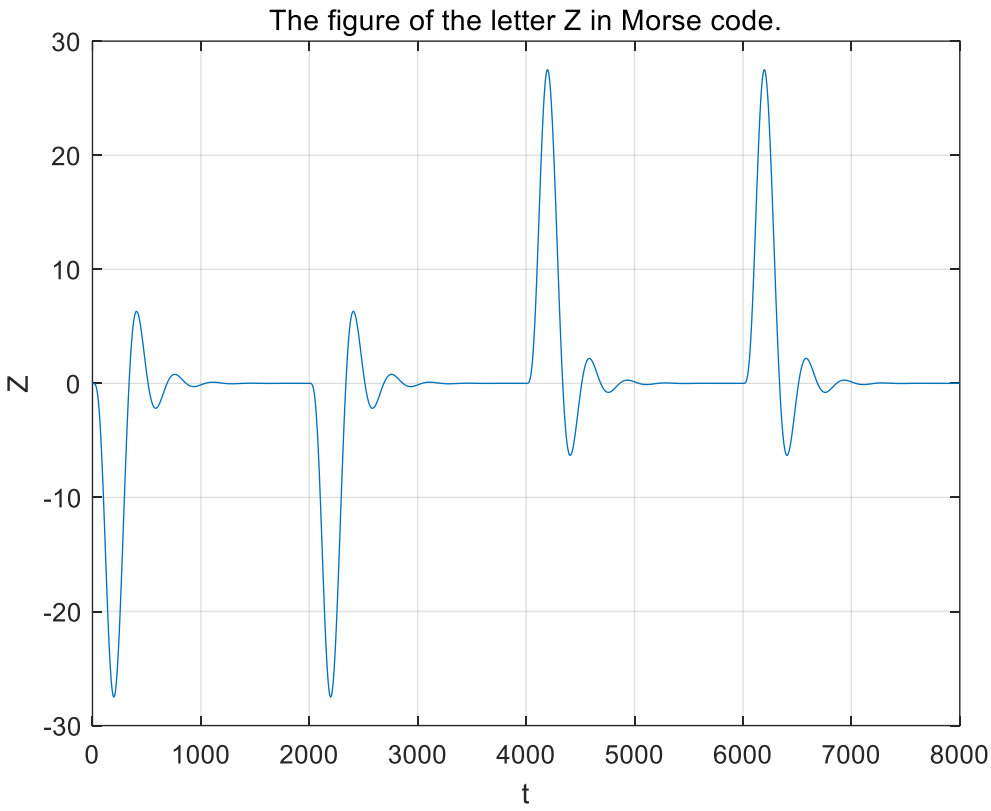
4. 6a) b)

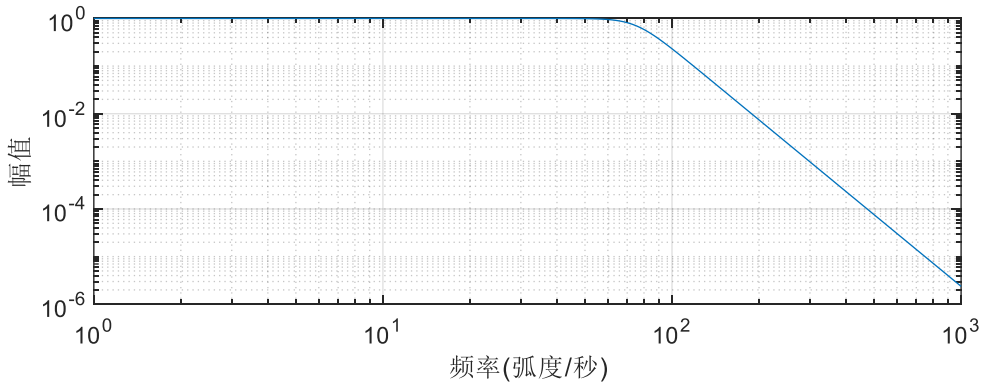
Code:

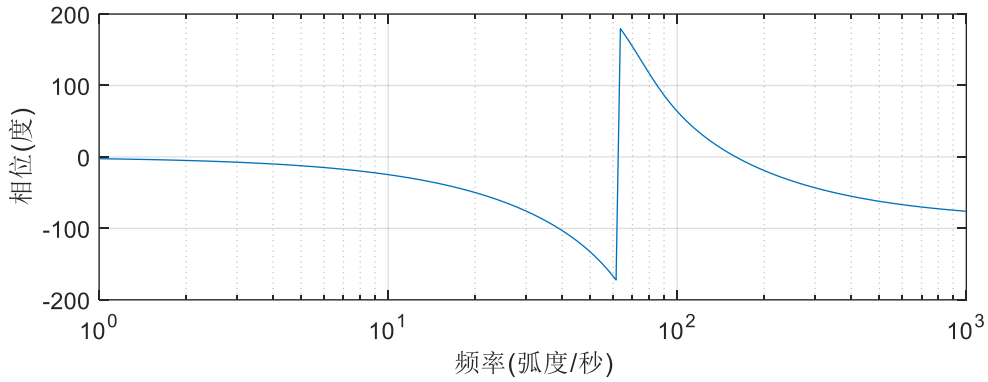
```
clc
clear all
load ctftmod.mat;
```

```
z=[dash dash dot dot];
figure(1);
plot(z);
xlabel('t');
ylabel('Z');
title('The figure of the letter Z in Morse code.');
grid on;
figure(2);
freqs(bf,af);
```

output:







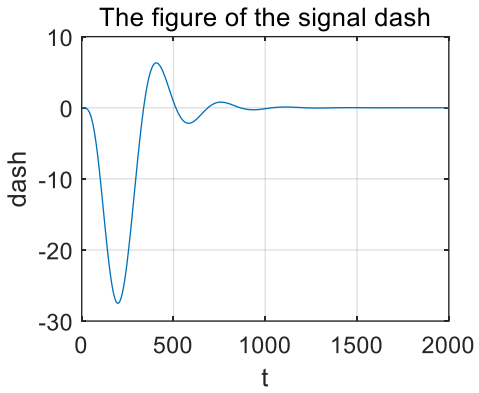
4.6.c)

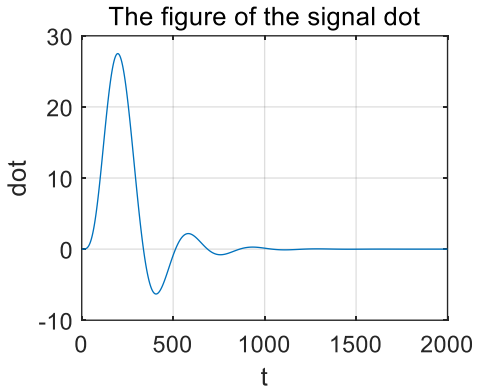
Code:

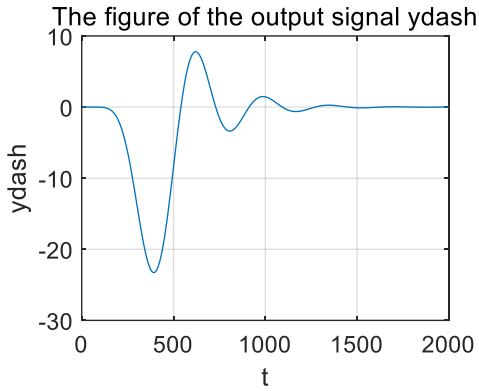
```
clc
clear all
load ctftmod.mat;
ydash=lsim(bf,af,dash,t(1:length(dash)));
ydot=lsim(bf,af,dot,t(1:length(dot)));
figure(1);
subplot(2,2,1);
plot(dash);
xlabel('t');
ylabel('dash');
title('The figure of the signal dash');
grid on;
subplot(2,2,2);
```

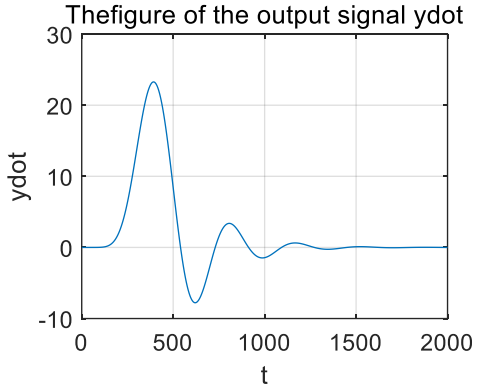
```
plot(dot);
xlabel('t');
ylabel('dot');
title('The figure of the signal dot');
grid on;
subplot(2,2,3);
plot(ydash);
xlabel('t');
ylabel('ydash');
title('The figure of the output signal ydash');
grid on;
subplot(2,2,4);
plot(ydot);
xlabel('t');
ylabel('ydot');
title('Thefigure of the output signal ydot');
grid on;
```

output:


The figure of the signal dash
dash
10
0
-10
-20
-30
0
500
1000
1500
2000
t


The figure of the signal dot
dot
30
20
10
0
-10
0
500
1000
1500
2000
t


The figure of the output signal ydash
ydash
10
0
-10
-20
-30
0
500
1000
1500
2000
t

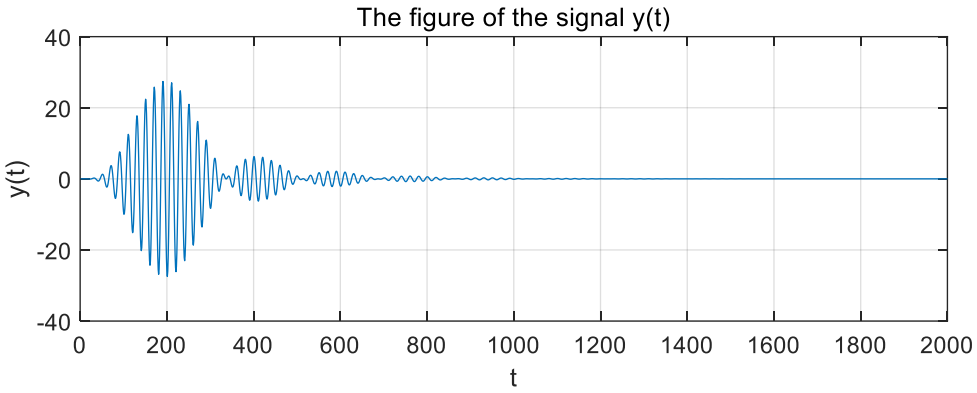

Thefigure of the output signal ydot
ydot
30
20
10
0
-10
0
500
1000
1500
2000
t

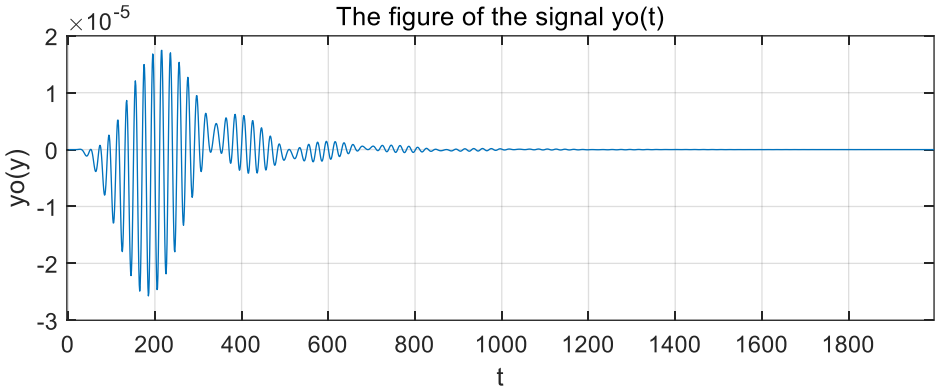
4. 6d)

Code:

```
clc
clear all
load ctftmod.mat;
y=dash.*cos(2*pi*f1*t(1:length(dash)));
yo=lsim(bf,af,y,t(1:length(dash)));
figure;
subplot(2,1,1);
plot(y);
xlabel('t');
ylabel('y(t)');
title('The figure of the signal y(t)');
grid on;
subplot(2,1,2);
plot(yo);
xlabel('t');
ylabel('yo(y)');
title('The figure of the signal yo(t)');
grid on;
```

output:





Explanation:

Key Insight:

The modulation y(t) shifts the dash signal to a higher frequency band, and the low-pass filter recovers the original signal envelope.

Relationship:

yo(t) represents the demodulated signal, which is a low-pass filtered version of y(t) that closely resembles the original dash signal.